

PRACTICAL NO 5

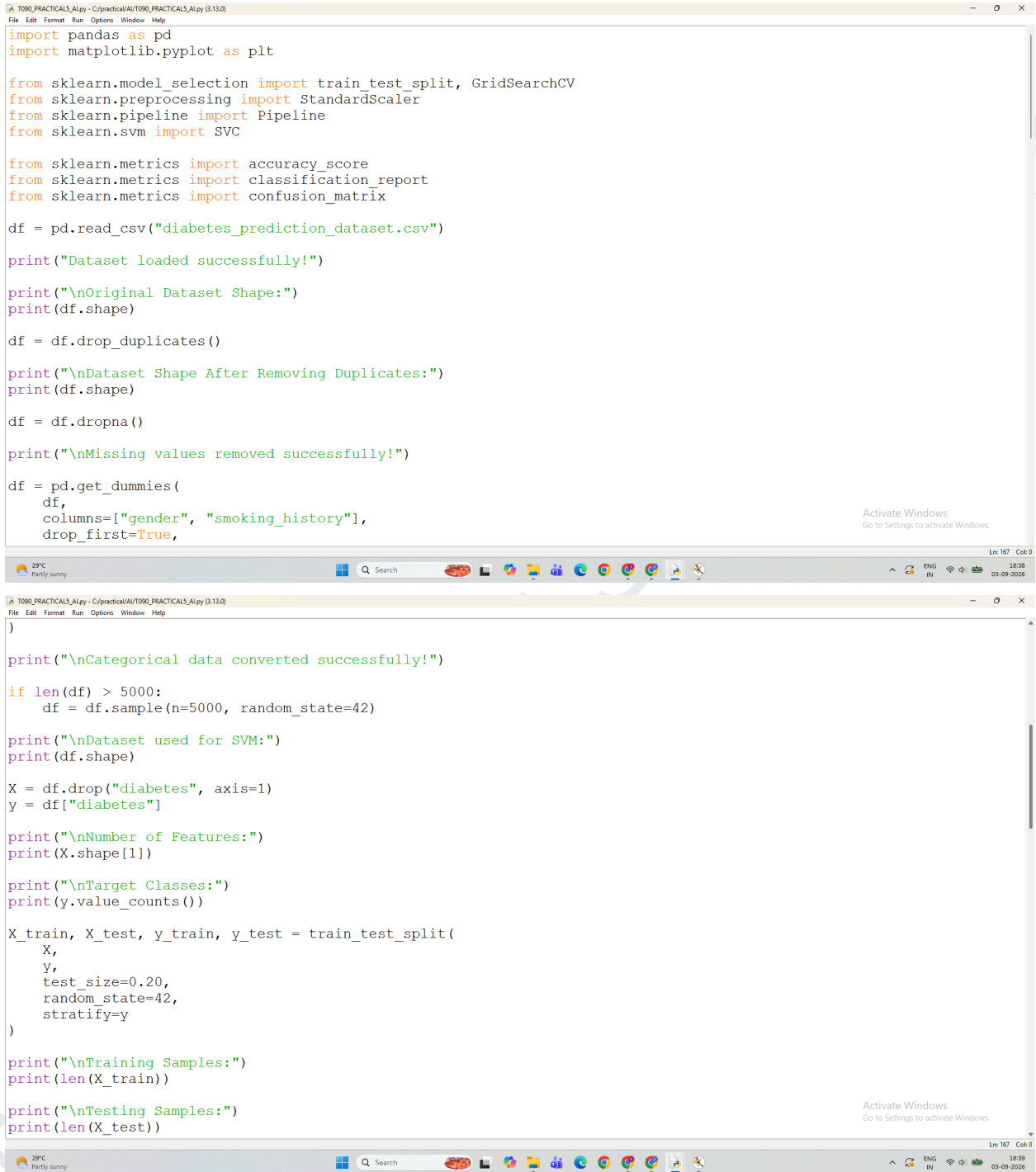
Aim :- Support Vector Machines (SVM)

1. Implement SVM algorithm for binary classification.
2. Train an SVM model using a given dataset and optimize its parameters.
3. Evaluate the performance of the SVM model on test data and analyze the results.

Step	Code / Function	Description
1	<code>pd.read_csv()</code>	Loads the Diabetes Prediction dataset.
2	<code>drop_duplicates()</code>	Removes duplicate records from the dataset.
3	<code>dropna()</code>	Removes records containing missing values.
4	<code>pd.get_dummies()</code>	Converts categorical features into numerical values.
5	<code>sample()</code>	Selects 5,000 records to reduce SVM memory usage.
6	<code>X and y</code>	Separates input features and the target <code>diabetes</code> .
7	<code>train_test_split()</code>	Divides the dataset into 80% training and 20% testing data.
8	<code>StandardScaler()</code>	Standardizes the features before SVM training.

9	<code>SVC()</code>	Creates the Support Vector Machine classification model.
10	<code>GridSearchCV()</code>	Tests different SVM parameters to find the best model.
11	<code>grid_search.fit()</code>	Trains and optimizes the SVM model using training data.
12	<code>best_params_</code>	Displays the best SVM parameters.
13	<code>predict()</code>	Predicts diabetes classes for the test data.
14	<code>accuracy_score()</code>	Calculates the model's prediction accuracy.
15	<code>classification_report()</code>	Displays precision, recall, and F1-score.
16	<code>confusion_matrix()</code>	Shows correct and incorrect predictions.
17	<code>plt.imshow()</code>	Visualizes the confusion matrix.
18	<code>plt.show()</code>	Displays the final graph.

CODE:



```

T090_PRACTICALS_AIpy - C:\practical\AI\T090_PRACTICALS_AIpy (3.13.0)
File Edit Format Run Options Window Help

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

df = pd.read_csv("diabetes_prediction_dataset.csv")

print("Dataset loaded successfully!")

print("\nOriginal Dataset Shape:")
print(df.shape)

df = df.drop_duplicates()

print("\nDataset Shape After Removing Duplicates:")
print(df.shape)

df = df.dropna()

print("\nMissing values removed successfully!")

df = pd.get_dummies(
    df,
    columns=["gender", "smoking_history"],
    drop_first=True,

)

print("\nCategorical data converted successfully!")

if len(df) > 5000:
    df = df.sample(n=5000, random_state=42)

print("\nDataset used for SVM:")
print(df.shape)

X = df.drop("diabetes", axis=1)
y = df["diabetes"]

print("\nNumber of Features:")
print(X.shape[1])

print("\nTarget Classes:")
print(y.value_counts())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:")
print(len(X_train))

print("\nTesting Samples:")
print(len(X_test))

```

```

T090_PRACTICALS_Alpy - C:\practical\AI\T090_PRACTICALS_Alpy (3.13.0)
File Edit Format Run Options Window Help
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC())
])

parameters = {
    "svm__C": [1, 10, 100],
    "svm__kernel": ["linear", "rbf"]
}

grid_search = GridSearchCV(
    pipeline,
    parameters,
    cv=3,
    scoring="accuracy",
    n_jobs=1
)

print("\n=====")
print("Optimizing SVM Parameters...")
print("Please wait...")
print("=====")

grid_search.fit(X_train, y_train)

print("\nBest Parameters:")
print(grid_search.best_params_)

print("\nBest Cross-Validation Accuracy:")
print(round(grid_search.best_score_ * 100, 2), "%")

best_model = grid_search.best_estimator_

T090_PRACTICALS_Alpy - C:\practical\AI\T090_PRACTICALS_Alpy (3.13.0)
File Edit Format Run Options Window Help
y_pred = best_model.predict(X_test)

print("\nPrediction completed successfully!")

accuracy = accuracy_score(y_test, y_pred)

print("\nTest Accuracy:")
print(round(accuracy * 100, 2), "%")

print("\nClassification Report:")

print(classification_report(
    y_test,
    y_pred,
    target_names=["No Diabetes", "Diabetes"],
    zero_division=0
))

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(7, 6))

plt.imshow(cm)

plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(

```

```

T090_PRACTICAL5_AI.py - C:/practical/AI/T090_PRACTICAL5_AI.py (3.13.0)
File Edit Format Run Options Window Help
plt.yticks(
    [0, 1],
    ["No Diabetes", "Diabetes"]
)

for i in range(2):
    for j in range(2):
        plt.text(
            j,
            i,
            str(cm[i, j]),
            ha="center",
            va="center"
        )

plt.colorbar()
plt.tight_layout()

plt.show()

print("\n=====")
print("SVM PRACTICAL COMPLETED")
print("=====")

print("\nFinal Accuracy:",
      round(accuracy * 100, 2), "%")

print("\nBest Parameters:")
print(grid_search.best_params_)

print("\n=====")

```

Activate Windows
Go to Settings to activate Windows.

29°C
Partly sunny

Search

Ln: 167 Col: 0

18:40
03-09-2026

OUTPUT:

```

Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/practical/AI/T090_PRACTICAL5_AI.py =====
Dataset loaded successfully!

Original Dataset Shape:
(100000, 9)

Dataset Shape After Removing Duplicates:
(96146, 9)

Missing values removed successfully!

Categorical data converted successfully!

Dataset used for SVM:
(5000, 14)

Number of Features:
13

Target Classes:
diabetes
0    4567
1     433
Name: count, dtype: int64

Training Samples:
4000

Testing Samples:

```

Activate Windows
Go to Settings to activate Windows.

29°C
Partly sunny

Search

Ln: 65 Col: 0

18:40
03-09-2026

```
=====
Optimizing SVM Parameters...
Please wait...
=====

Best Parameters:
{'svm_C': 10, 'svm_kernel': 'linear'}

Best Cross-Validation Accuracy:
95.85 %

Prediction completed successfully!

Test Accuracy:
96.0 %

Classification Report:
              precision    recall  f1-score   support

No Diabetes      0.96      1.00      0.98       913
Diabetes         0.94      0.57      0.71        87

   accuracy      0.96      0.79      0.96      1000
  macro avg      0.95      0.79      0.85      1000
 weighted avg      0.96      0.96      0.96      1000

Confusion Matrix:
[[910   3]
 [ 37  50]]
```

Activate Windows
Go to Settings to activate Windows.

GRAPH:

